UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

Appunti di
Linguaggi di Programmazione I

Anno 2021

Valentino Bocchetti

Contents

1 Introduzione al corso e informazioni **3**

1.1 Esame 3

2 Suddivisione del corso **3**

2.1 Parte I 3

2.2 Parte II (modello object-oriented, Java) 3

2.3 Parte III (linguaggi funzionali, ML) 4

2.4 Parte IV (linguaggi logici, Prolog) 4

3 Lezione del 09-03 **5**

3.1 Breve storia 5

3.2 Definizione di base 6

3.3 Linguaggi completi 6

3.4 Macchine astratte 7

3.5 Traduttori del linguaggio 9

3.6 SRT 11

3.7 Compilazione ed esecuzione 11

3.8 Proprietà dei linguaggi 13

3.9 Criteri di scelta dei linguaggi 13

4 Lezione del 12-03 **13**

4.1 Paradigmi computazionali 13

4.1.1 Paradigma imperativo + esempio (pseudo-linguaggio + c): 14

4.1.2 Paradigma funzionale + esempio (pseudo-linguaggio + lisp): 15

4.1.3 Paradigma logico + esempio (prolog): . . . 15
4.1.4 Imperativo vs funzionale . . . 16
4.1.5 Orientato ad oggetti . . . 16
4.1.6 Parallelo . . . 17
4.2 Modello Imperativo nel dettaglio . . . 17
4.3 Esempi di semantica di assegnamenti in C . . . 19
4.3.1 Assegnazione . . . 19
4.3.2 Operatore & . . . 19
4.3.3 Operatore & a sinistra . . . 19
4.3.4 Puntatori (a destra) . . . 19
4.3.5 Puntatori (a sinistra) . . . 20
4.3.6 vettori e puntatori . . . 20

5 Lezione del 16-03 **22**
5.1 Data Object e legami . . . 22
5.1.1 Variazione dei legami . . . 22
5.1.2 Legame di tipo . . . 24
5.1.3 Type checking . . . 25
5.1.4 Il linguaggio perfetto . . . 25
5.1.5 Strategie . . . 26
5.2 Blocchi di istruzioni . . . 26
5.2.1 Ambito di validità di legami . . . 27
5.2.2 Legami di nome . . . 27
5.2.3 Mascheramento . . . 28

5.2.4 Legami di locazione 28

5.2.5 Stack di attivazione 29

6 Lezione del 19-03 **30**

6.1 Procedure come astrazioni 30

6.2 Record di attivazione 31

6.2.1 Ambiente locale 31

6.2.2 Propagazione dei Data OBJECT 31

6.3 Parametrizzazione di procedure 34

6.3.1 Parametri IN 35

6.3.2 Parametri OUT 36

6.3.3 Parametri IN-OUT 36

6.4 Aliasing 36

6.5 Procedure come parametri di procedure 36

7 Lezione del 23-03 **37**

7.1 Memo 37

7.2 Funzioni 37

8 Lezione del 26-03 **37**

8.1 Esempi sul record di attivazione nei linguaggi 37

8.1.1 Esempio interferenza 37

8.1.2 Esempio copia 38

8.2 Macro 38

8.3 Implemantazione efficiente dello scoping statico 39

8.4 Annidamento 39

8.5 Rappresentazione variabili non locali 40

8.6 Mantenimento del vettore degli ambienti non locali 41

8.7 Proprietà di questa implementazione 41

8.8 Paradigma OO 42

8.9 Encapsulation 42

8.10 C++ vs Java 42

8.11 UML 42

8.12 Essenza di un oggetto 42

8.13 Classe 43

8.14 Polimorfismo 43

9 Lezione del 30-03 **43**

9.1 Classi astratte 43

9.2 Visibilità delle componenti di una classe 43

9.3 Ereditarietà 44

9.3.1 Generalizzazione 44

9.3.2 Specializzazione 44

9.4 Associazioni e molteplicità 44

9.4.1 Associazioni complesse 44

9.4.2 Ruoli nelle associazioni 44

9.4.3 Associazioni riflessive 45

9.5 Introduzione a Java 45

9.5.1 Errori comuni 45

9.5.2 Compilazione ed Esecuzione 46

9.5.3 Sicurezza 46

9.5.4 Costruttori 47

1 Introduzione al corso e informazioni

`Email` → pab@unina.it

`Ricevimento` → da richiedere in precedenza con una mail

1.1 Esame

- Uno scritto java + altre parti del corso;
- In caso di dubbi c'è un orale;

Lo scritto può essere mantenuto per la durata di un anno.

2 Suddivisione del corso

2.1 Parte I

- Storia e concetti di base;
- Primi cenni ai paradigmi dei linguaggi di programmazione;
- Il modello imperativo;

2.2 Parte II (modello object-oriented, Java)

- Studio dei costrutti più avanzati
  - qualificatori di classi;
  - metodi e attributi;
  - classi astratte;
  - interfacce;
  - template;
  - classi interne;
  - gestione degli errori;
  - eccezioni;

2.3 Parte III (linguaggi funzionali, ML)

- ML e costrutti avanzati;
- funzioni di ordine superiore;
- polimorfismo;
- tipi di dato astratti e interfacce;
- template;
- gestione degli errori;
- eccezioni;
- type inference;

2.4 Parte IV (linguaggi logici, Prolog)

- Costrutti fondamentali
  - termini;
  - predicati;
  - regole;
- Tecniche di programmazione avanzate
  - invertibilità;
  - nondeterminismo;

3 Lezione del 09-03

3.1 Breve storia

Il primo linguaggio introdotto è il FORTRAN (introdotto con lo scopo di calcoli scientifici). Nasce la concezione di variabile

Anno	Linguaggio
1954	FORTRAN (FORmula TRANslation)
1960	COBOL
	ALGOL (60 e 68)
	`PL/1`
	Simula 67
	PASCAL
	LISP
	APL
	BASIC
`1970/1980`	PROLOG
	SMALLTALK
	C
	`MODULA/2`
	ADA

- **Cobol** → orientato ai database (introduce i record);
- **Agol60** → indipendenza dalla macchina e definizione mediante grammatica (supporto dell'iterazione e ricorsione, introduzione delle strutture a blocco);
- **Lisp** → primo vero linguaggio di manipolazione simbolica, linguaggio funzionale (non esiste lo statement di assegnazione);
- **Prolog** → primo (e principale) linguaggio di programmazione logica, porta l'invertibilità e una programmazione in stile nondeterministico (essenzialmente non tipato; queste vengono ottenute mediante metaprogrammazione);
- **Simula 67** → classe come incapsulamento di dati e procedure, istanze delle classi (oggetti);
- `PL/1` abilità ad eseguire procedure specificate quando si verifica una condizione eccezionale; (multitasking);
- **Pascal** → programmazione strutturata, tipi di dato definiti da utente (ricchezza di strutture dati); nessuna encapsulation.

3.2 Definizione di base

Linguaggio di programmazione

È un linguaggio che è usato per esprimere (mediante un programma) un processo con il quale un processore può risolvere un problema.

Processore

È la macchina che eseguirà il processo descritto dal programma (da non intendere come un singolo oggetto, ma coome una *architettura di elaborazione*).

Programma

È l'espressione codificata di un processo.

3.3 Linguaggi completi

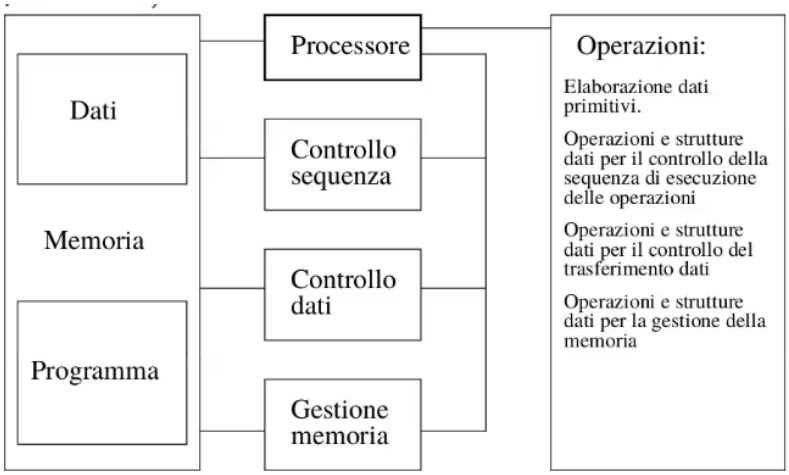
Sono linguaggi in grado di programmare qualunque funzione calcolabile (*general purpose*). Tecnicamente devono poter simulare qualunque *macchina di Turing.*

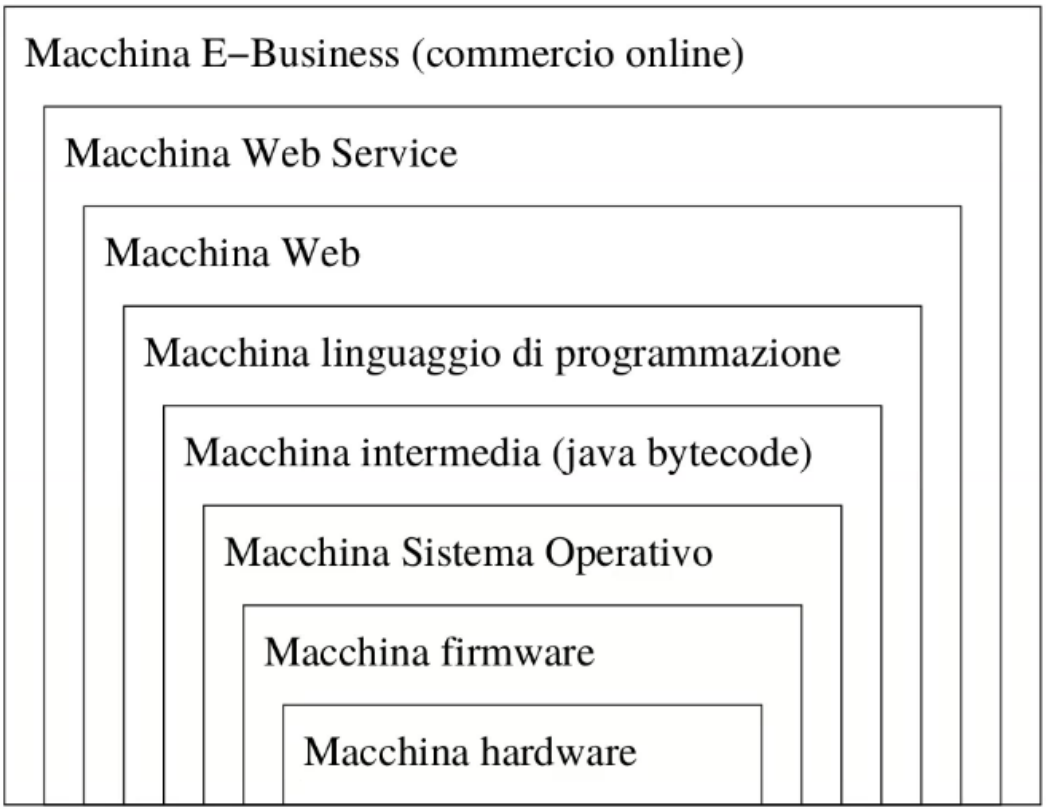
Sono completi solo quelli che riescono ad esprimere anche programmi di cui non è decidibile la terminazione. Per dimostrare la completezza di un linguaggio lo si usa per simulare arbitrarie macchine di Turing.

Ad esempio non sono linguaggi completi **SQL** (perchè la terminazione dei programmi è sempre decidibile; è spesso immerso in linguaggio completo) e **HTML**.

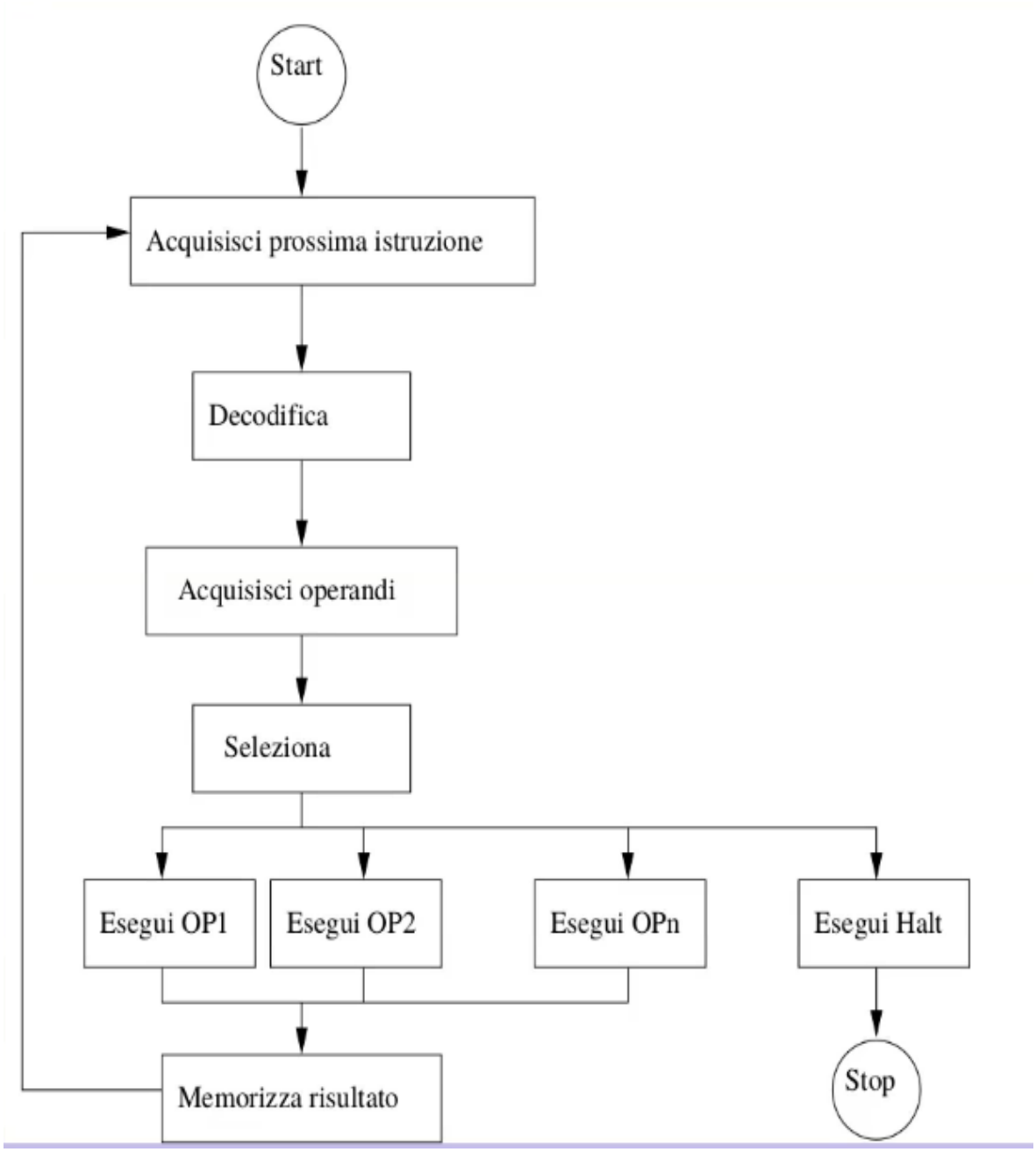
3.4 Macchine astratte

Dato un linguaggio di programmazione L, una macchina astratta per L (in simboli `M_L`) è un qualsiasi insieme di strutture dati e algoritimi che permettano di memorizzare ed eseguire programmi scritti in L. La struttura di una macchina astratta è (essenzialmente memoria e processore):





Processore della macchina astratta



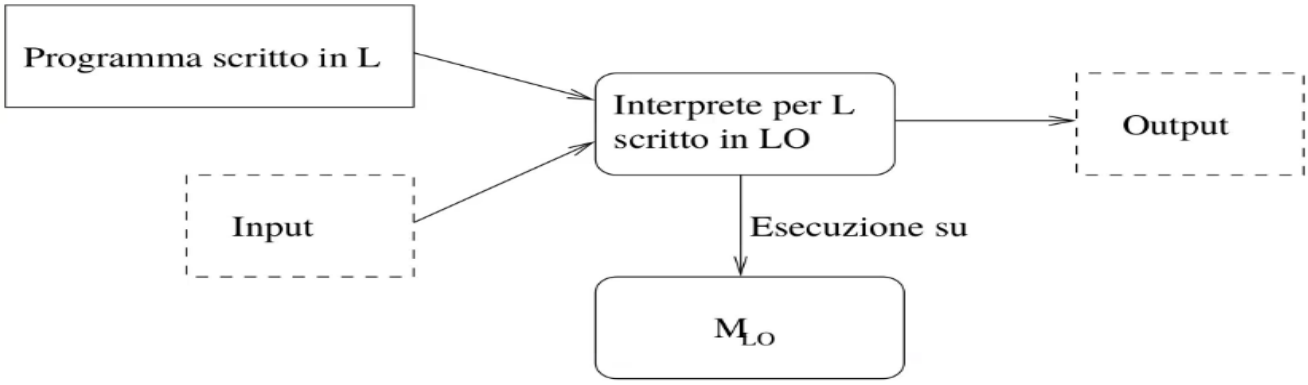
Nel tempo si è pensato a vari modi di implementarlo:

- Hardware (es per *Lisp* e *Prolog*), ma abbandonato rapidamente;
- Firmware → soluzione più flessibile e semplice (ed economico);
- Software → come nel caso della JVM;

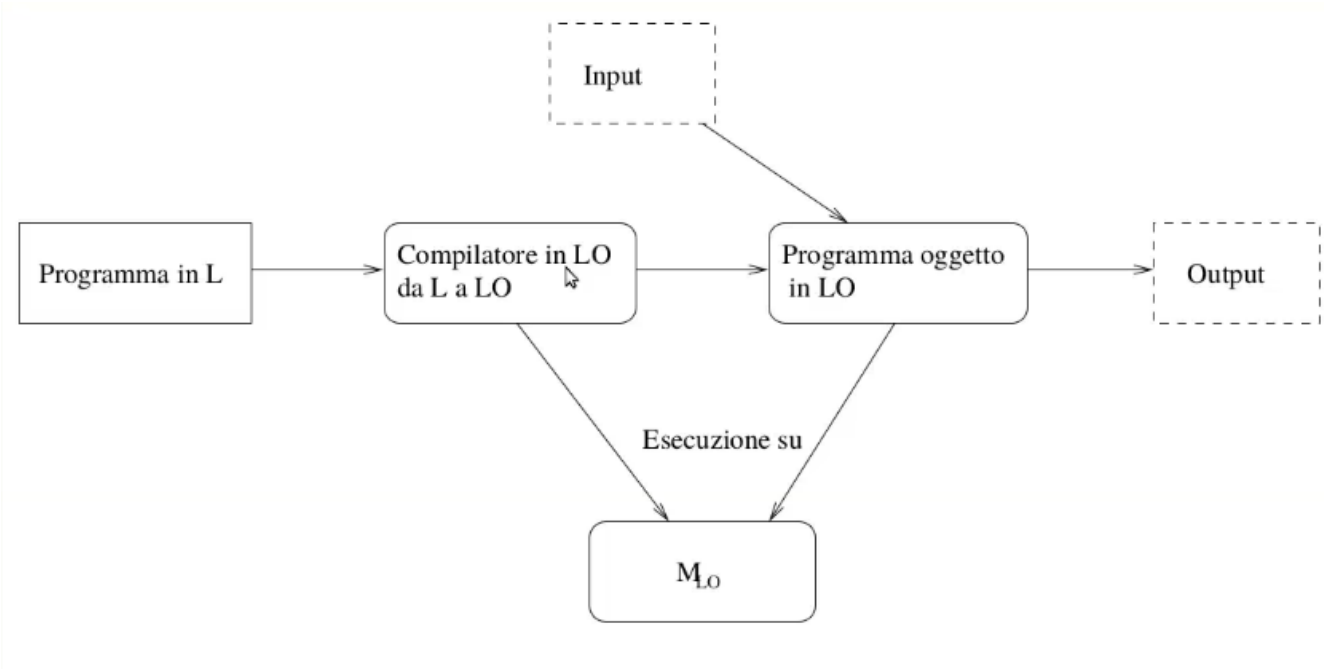
3.5 Traduttori del linguaggio

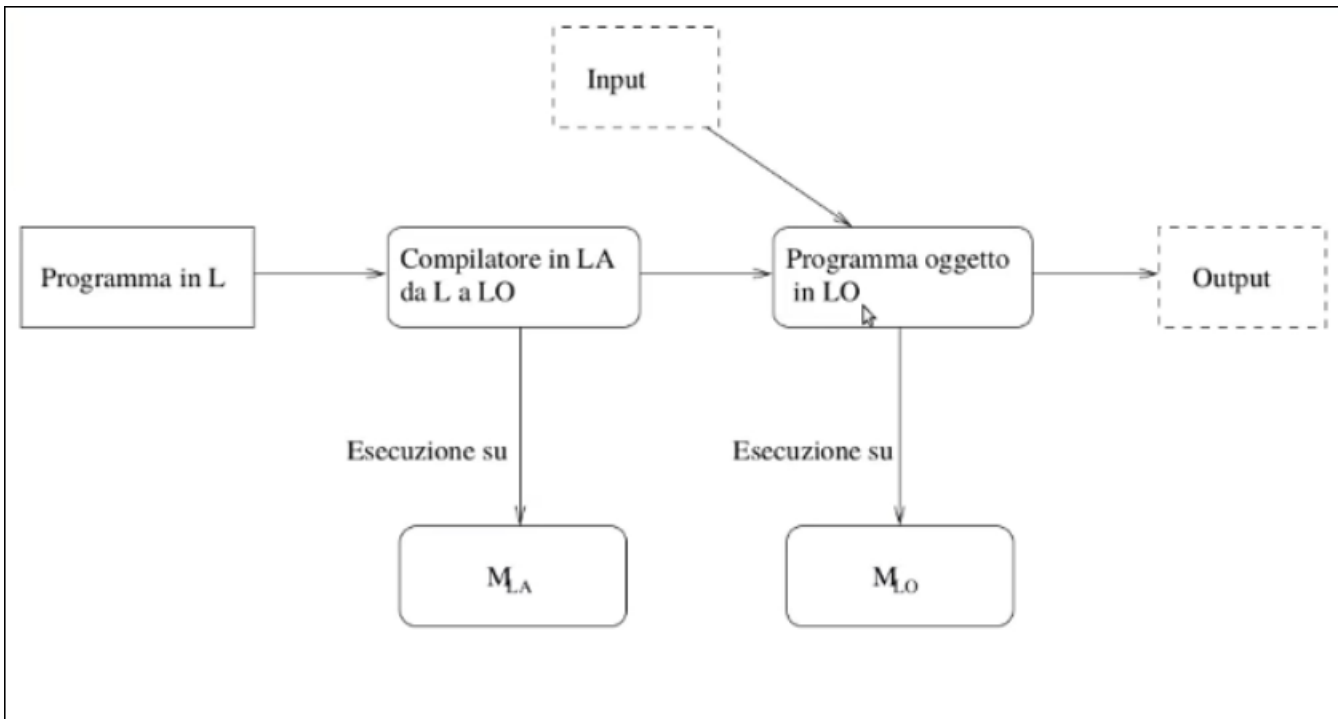
- **Interpreti** → traducono ed eseguono un costrutto alla volta
  - PRO : debug;
- **Compilatori** → prima traducono l'intero programma, che poi può essere eseguito
  - PRO : velocità di esecuzione finale;
  - PRO : più controlli e in anticipo.

Interpretazione pura



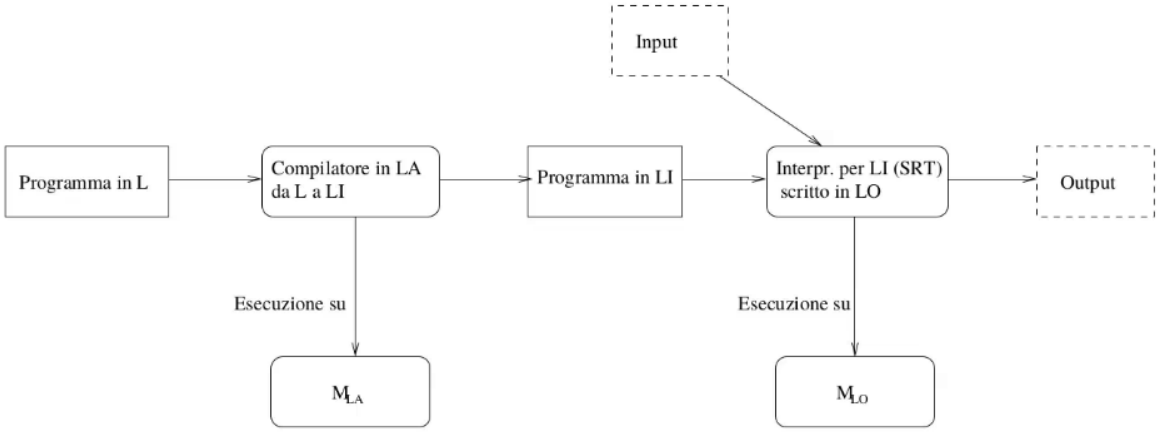
Compilazione pura (caso semplice e generale)





Nella compilazione pura può accadere che L ed LO siano uguali (il vantaggio è il riutilizzo del compilatore $\rightarrow$ **custom compilation**)

Compilazione per macchina intermedia



($SRT \rightarrow$ supporto al run time)

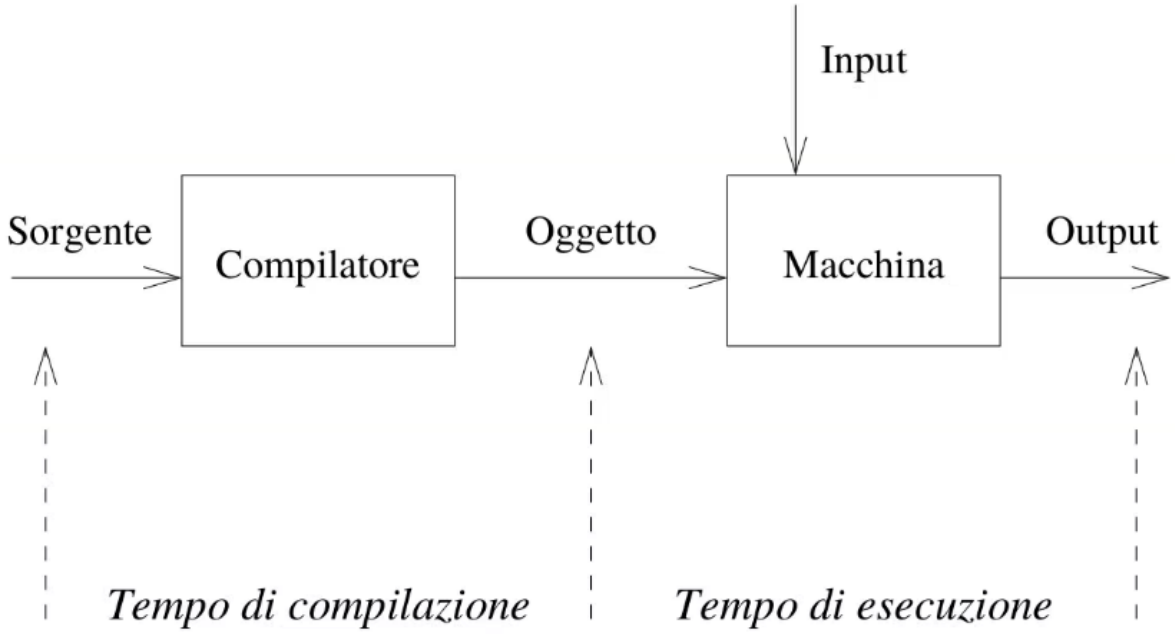
Torna utile nel caso di linguaggi molto diversi dalla macchina astratta sottostante. Invece di creare hardware apposito creiamo un interprete che lo sostituisca

3.6 SRT

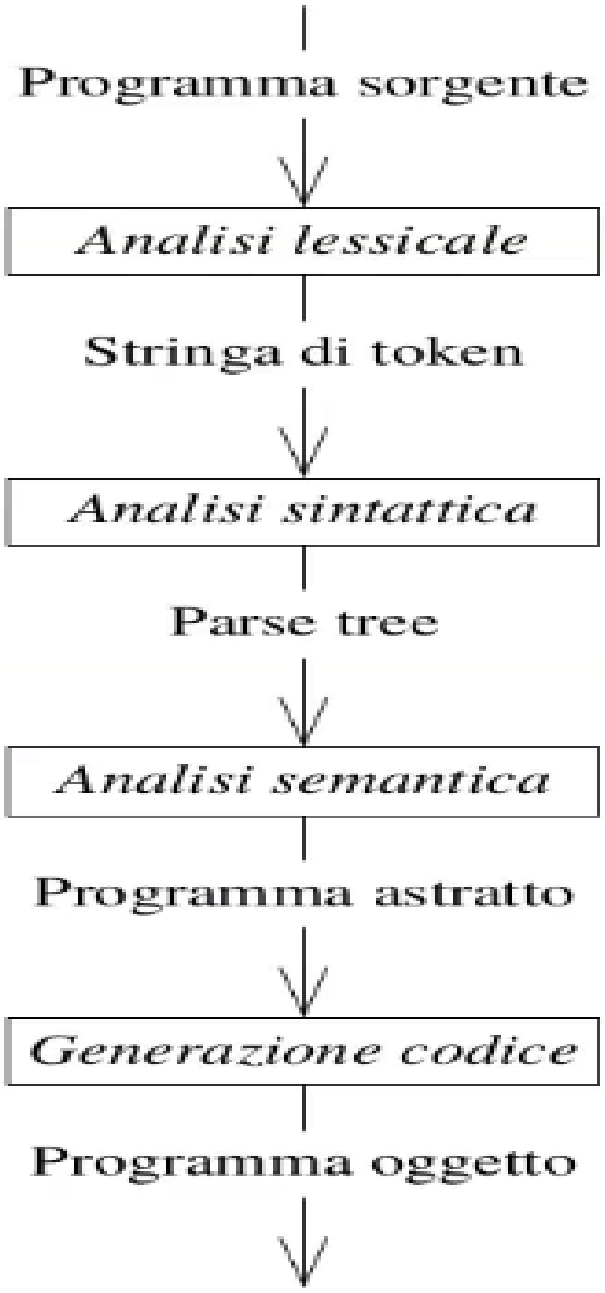
- Funzionalità aggiuntive (rispetto alla macchina astratta sottostante);
- Funzioni di basso livello / interfacce con il S.O. (es `I/O`);
- Gestione della memoria;
- Garbage collection;
- Gestione dell'heap;
- Gestione dello stack;

Non è necessariamente una macchina astratta radicalmente diversa, ma a volte solo un pacchetto di funzioni o librerie aggiunte automaticamente al codice oggetto.

3.7 Compilazione ed esecuzione



Fasi della compilazione



(Analisi lessicale e sintattica sono stati un problema → venivano fatte in maniera artigianale)

3.8 Proprietà dei linguaggi

- **Semplicità** → (coincisione) VS (leggibilità)
  - **Semantica** → minimo numero di concetti e strutture;
  - **Sintattica** → unica rappresentabilità di ogni concetto;
- **Astrazione** → rappresentare solo attributi essenziali
  - **Dati** → nascondere i dettagli di oggetti;
  - **Procedure** → facilitare la modularità del progetto;
- **Espressività** → (facilità di rappresentazione di oggetti) VS (semplicità);
- **Ortogonalità** → meno eccezioni alla regole del linguaggio;
- **Portabilità.**

3.9 Criteri di scelta dei linguaggi

- Disponibilità dei traduttori;
- Maggiore conoscenza da parte del programmatore;
- Esistenza di standard di portabilità (es ADA);
- Comodità dell'ambiente di programmazione;
- Sintassi aderente al problema;
- Semantica aderente alla architettura fisica non importante (non è più un criterio stringente).

4 Lezione del 12-03

4.1 Paradigmi computazionali

Il paradigma di appartenenza può influenzare radicalmente il modo in cui si risolve il problema, ma non è l'unica discriminante. Infatti sono discriminanti anche:

- Il sistema di tipi supportato;
- Eventuale supporto alle eccezioni;
- Modello di concorrenza e sicronizzazione.

Sono presenti tre tipi di paradigmi computazionali:

- Imperativo;
- Funzionale;
- Logico.

es. Un programma specifica sequenze di modifiche da apportare allo stato della macchina (memoria)

Vogliamo scrivere in tutti e tre i linguaggi la funzione membro (X, L), che decida se l'elemento X appartiene alla lista L

```
membro(2, [1,2,3]) = true; membro(2, [1,2,3]) = false;
```

A questo scopo supponiamo siano già esistenti le funzioni:

- vuota(L), che restituisce true se L è vuota, false altrimenti;
- testa(L), che restituisce il primo elemento della lista L;
- coda(L), che restituisce una sottolista ottenuta rimuovendo il primo elemento di L

4.1.1 Paradigma imperativo + esempio (pseudo-linguaggio + c):

```
procedure membro (X,L)
    local L1 = L
    while not vuota(L1) and not X=testa(L1)
        do L1 = coda(L1)
return not vuota(L1)
```

```
bool member(X, L){
    List L1 = L;
    while ( ! empty(L1) && ! X=testa(L1))
        L1 = coda(L1);
    return (! vuota(L1));
}
```

4.1.2 Paradigma funzionale + esempio (pseudo-linguaggio + lisp):

Il programma e le sue componenti sono funzioni. Esecuzione come valutazioni di funzioni

```
function member(X, L)
if vuota (L) then false
else if X == testa(L) then true
else member(X, coda (L))
```

```
(defun membro (x l)
    (cond ((null l) nil)
        ((equal x (first)) T)
        (T (membro x (rest l)))))
```

Notiamo che anche il c può prestarsi alla versione funzionale

```
bool member(X, L){
    return (vuota (L)) false :
        (X == testa(L))? true :
        member(X, coda(L))}
```

4.1.3 Paradigma logico + esempio (prolog):

Programmma come descrizione logica di un problema. Esecuzione analoga a processi di dimostrazione di teoremi

```
membro(X, [X|L]) .
membro(X, [Y|L]) : - membro(X, L).
```

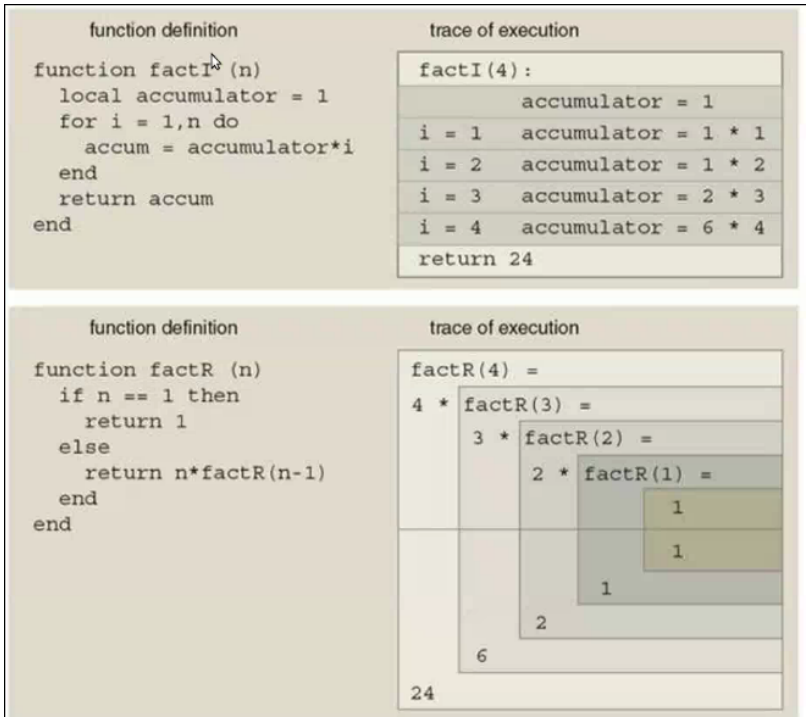
Notiamo come i parametri formali di member possono essere pattern (consistono di definizioni di predicati). Esecuzione:

`member(2,[1,2,3]) restituisce yes (true)` `member(0,[1,2,3]) restituisce no (false)`

Presenta diverse proprietà:

- Si comporta come un generatore → query con variabili: member(X,[1,2,3]) restituisce
  - X=1;
  - X=2;
  - X=3;
  - no;
- Propone risposte con variabili: member(1, L) restituisce
  - `L=[1| L_0], L=[Y_0, 1 | L_1]; L = [Y_0, Y_1, 1 | L_2] ...`
- Invertibilità → nessuna distinzione tra input e output → un solo predicato (programma), molte funzioni;

4.1.4 Imperativo vs funzionale



Si noti l'eleminazione degli assegnamenti:

- n rimpiazza i (ricorsione invece di cicli)

4.1.5 Orientato ad oggetti

Programma costituito da ogetti che scambiano messaggi.

4.1.6 Parallelo

Programmi che descrivono entità distribuite che sono eseguite contemporaneamente e in modo asincrono.

4.2 Modello Imperativo nel dettaglio

Consiste in un insieme di "contenitori di dati":

- Ad es. parole (o celle) della memoria centrale;
- Tipicamente rappresentate dal loro indirizzo

Concettualmente la memoria quindi è:

- Una funzione da uno spazio di locazioni ad uno spazio di valori;
- mem(loc) = "valore contenuto in loc"

La sintassi è del tipo

`<assegnazione>` ::= `<name>` `<assigment-operator>` `<expression>`

dove:

- `<name>` rappresenta la locazione dove viene posto il risultato;
- `<expression>` vengono specificati una computazione e i riferimenti ai valori necessari alla computazione

In Pascal:

```
a := b + c;
```

Il valore di `<expression>` va memorizzato nell'indirizzo rappresentato da `<name>`. Il valore di `<expression>` dipenderà dai valori contenuti negli indirizzi degli argomenti di `<expression>` rappresentati dai nomi di questi ottenuto seguendo le prescrizione del codice associato al suo nome

Questo tipo di modello è il più vicino agli elaboratori "standard". I programmi sono descrizioni di sequenze di modifiche della "memoria" del calcolatore (come nel caso del modo di pilotare i display, salvare su memorie periferiche). Ogni esecuzione consiste di 4 passi (in sostanza un assrgnamento):

- Ottenere indirizzi delle locazioni di operandi e risultato;
- Lettura degli operandi;
- Valutazione del risultato;
- Memorizzazione del risultato

In questo tipo di modello le variabili di un programma costituiscono una memoria, mentre i parametri formali delle `funzioni/procedure` no (sia concettualmente che per come sono realizzati). L'ambiente (environment) comprende:

- Un insieme di nomi di variabili e parametri
  - non indirizi di memoria, piuttosto identificatori, associati al loro valore (direttamente e non)

Nel paradigma imperativo, la funzione env (ambiente) associa gli identificatori a locazioni di memoria, le quali, a loro volta, sono associate (funzione mem) al contenuto di memoria. Il valore di una variabile x è mem(env(x)). In questo tipo di paradigma la funzione env identifica una associazione immutabile (la locazione di memoria associata a un nome non cambia)

Nel paradigma funzionale, non esiste la funzione mem e la funzione env associa direttamente gli identificatori al valore della memoria

ATTENZIONE: env(x) è immutabile finchè esiste x (gli identificatori nascono e muoiono durante l'esecuzione)

In una assegnazione del tipo `x := x +1;` vediamo che il valore di destra va salvato nella variabile di sinistra:

- la x di sinistra indica la locazione associata al nome (cioè env(x));
- la x di destra indica il valore della memoria (cioè mem(env(x)))

4.3 Esempi di semantica di assegnamenti in C

4.3.1 Assegnazione

```
x=y
```

In questo caso avremo:

- sx:(env(x));
- dx: mem(env(y));

4.3.2 Operatore &

```
int *x; y
x = &y;
```

In questo caso avremo:

- sx: env(x);
- dx: env(y);

4.3.3 Operatore & a sinistra

```
&y = x;
```

`NB: ERRORE!` → sarebbe come scrivere env(env(y)) che è malformato (env accetta solo cose che siano identificatori)
Il C riporta errore di sintassi *not an lvalue*. Valori corretti possono essere:

- nome di variabile;
- `*(<pointer-expression>)`;

4.3.4 Puntatori (a destra)

```
y = *x;
```

In questo caso avremo:

- sx: env(x);
- dx: mem(mem(env(y)));

4.3.5 Puntatori (a sinistra)

In C `*x` indica la cella di memoria puntata da x (la cella di memoria il cui indirizzo sta scritto dentro x)

```
*x = y;
```

In questo caso avremo:

- sx: mem(env(x));
- dx: mem(env(y))

4.3.6 vettori e puntatori

```
int v[3];
```

In questo caso env(v) indica la prima cella di memoria del vettore v

y = v[1];

In questo caso avremo:

- sx: env(y);
- dx: mem(env(v) +1);

v[1] = y;

In questo caso avremo:

- sx: env(v) +1;
- dx: mem(env(y));

y = x[1] + v[1]; //puntatore + vettore

In questo caso avremo:

- sx: env(v) +1;

- dx: mem(men(env(x) +1)) + mem(env(v) + 1);

```
x[1] = y
```

In questo caso avremo:

- sx: mem(env(x)) +1

```
y = *x + *v
```

In questo caso avremo:

- dx: mem(mem(env(x))) + mem(env(v))

```
*v = y;
```

In questo caso avremo:

- sx: env(v); (visto come v[0] sarebbe env(v) + 0)

```
y = *(v+2)
```

In questo caso avremo:

- dx: mem(env(v) + 2);

```
y = (*v) + 2
```

In questo caso avremo:

- dx: mem(env(v)) + 2;

```
*(x+2) = ...
```

In questo caso avremo:

- sx: mem(env(x))+2

5 Lezione del 16-03

5.1 Data Object e legami

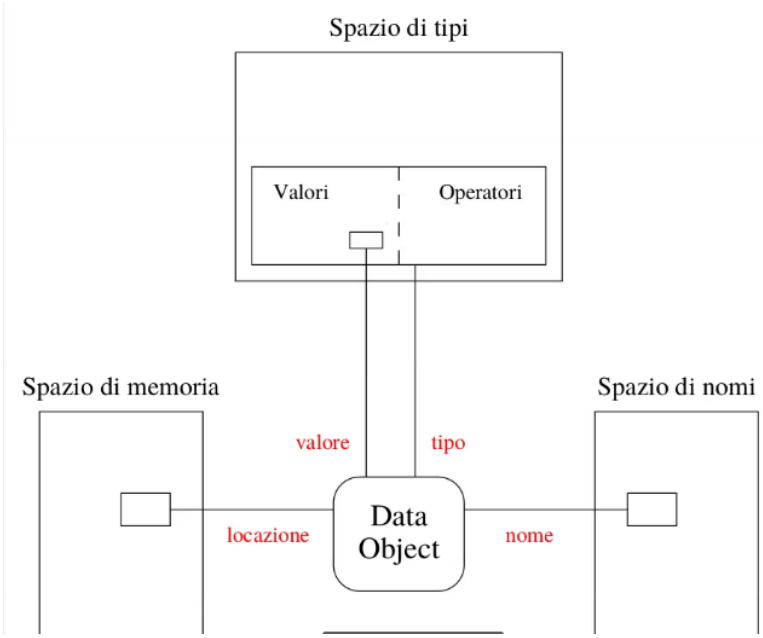
Ogni variabile, parametro, oggetto può essere rappresentato da un Data Object.

È una quadrupla (L, N, V, T) → locazione, nome, valore, tipo.

Legame → determinazioni di una delle componenti

Nel paradigma imperativo:

- Il legame di locazione corrisponde alla funzione *env*;
- il legame di valore di una variabile di nome x corrisponde a *mem(env(x))*.



5.1.1 Variazione dei legami

Possono avvenire:

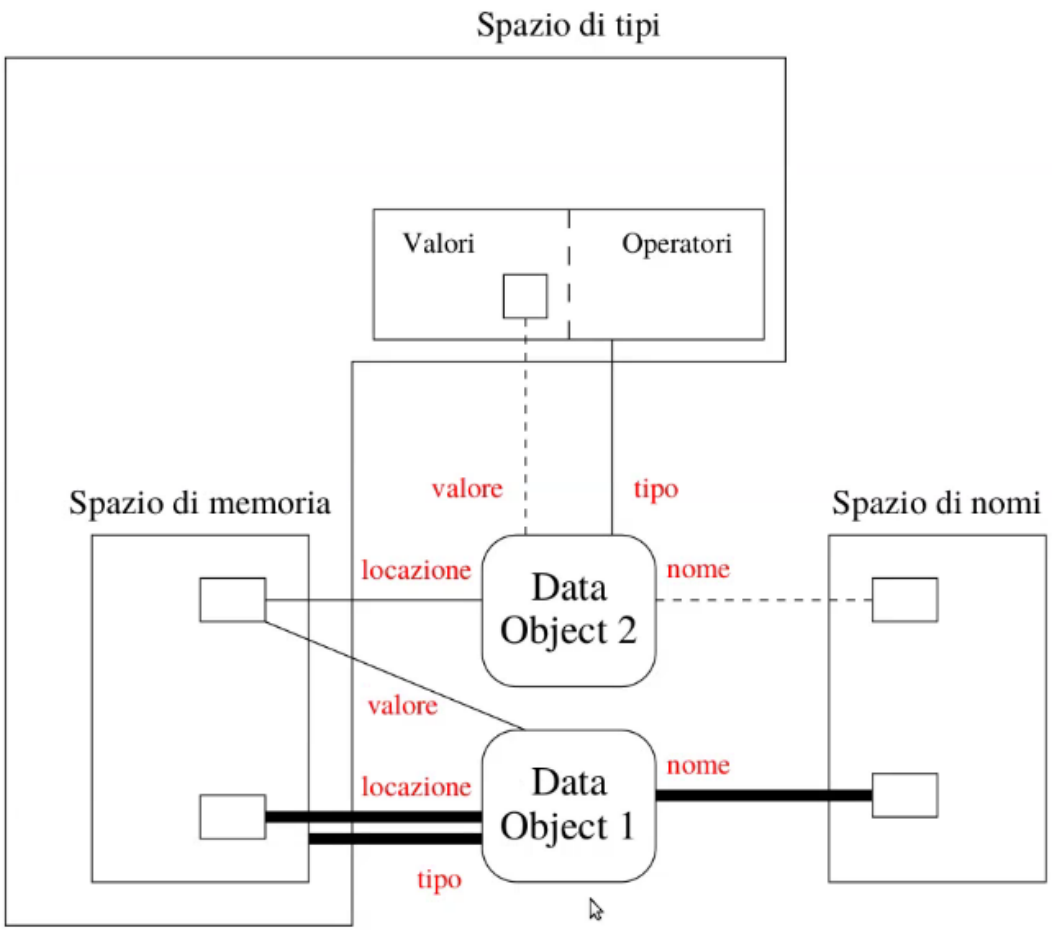
- Durante la compilazione (compile-time);
- Durante il caricamento in memoria (load-time);
- Durante L'esecuzione (run-time).

Location binding → avviene o durante il caricamento in memoria o a run-time

Name binding → avviene durante la compilazione, nell'istante in cui il compilatore incontra una dichiarazione

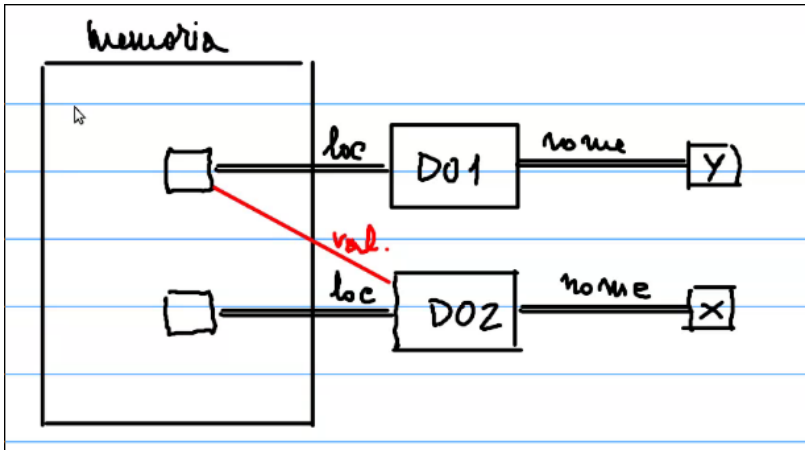
Type binding → avviene di solito durante la compilazione (nel momento in cui il compilatore incontra una dichiarazione); un tipo è definito dal sottospazio di valori che un data object puà assumere

1. I puntatori



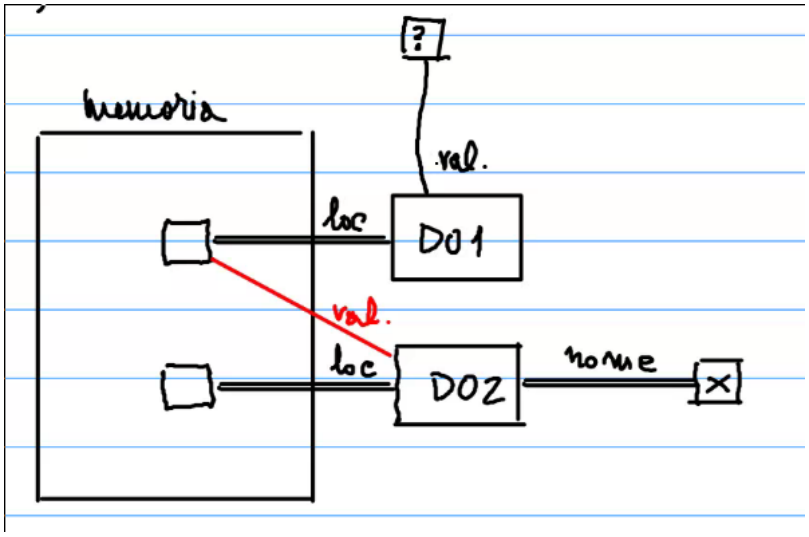
es

```
int y, *x; // CREA 2 DATA OBJECT
```




```
x = (int *) malloc(sizeof(int));
```

Il cast (int *) serve a CONTROLLA REC



2 data object coinvolti. Il secondo può non avere un legame di nome o di valore. La deallocazione è necessaria, perchè la modifica del legame di valore genera di solito dati non più accessibili per nome o riferimento

Alcuni linguaggi possiedono meccanismi di recupero automatico di memoria

5.1.2 Legame di tipo

Correlato al legame di valore. Ogni volta che il legame viene modificato occorre controllare la consistenza con il legame di tipo.

Un linguaggio è *dinamicamente tipato* se il legame (e le variazioni di legame) e di conseguenza anche il controllo di consistenza avvengono durante l'esecuzione (*es* linguaggi di scripting e Python).

Un linguaggio è *staticamente tipato* se il legame avviene durante la compilazione; in questo caso il controllo di consistenza può avvenire in entrambe le fasi.

5.1.3 Type checking

Mecanismo di controllo di consistenza della coppia valore-tipo. Avviene:

- Durante la compilazione;
- Durante l'esecuzione;

Un linguaggio è *fortemente tipato* se il controllo di consistenza avviene sempre (*es* Java).

Un linguaggio è *debolmente tipato* se il controllo di consistenza può non avvvenire affatto in numerosi casi (*es* C). Un esempio in C sono le union:

```
#include <cstdio>
union numero{
  int numero_piccolo;
  double numero_grande;
} numeri;

int main(){
  numeri.numero_grande = 3.0;
  printf("%d\n", numeri.numero_piccolo);
  printf("%f\n", numeri.numero_grande);
}
```

0
3.0

Si inserisce un double ma si legge come fosse un int → viene stampato 0 (il compilatore non segnala l'errore).

Essendo esposte a questi tipi di errori si tende a non usarle.

5.1.4 Il linguaggio perfetto

Sarebbe conveniente se il type checking avvenisse completamente durante la compilazione e in cui il compilatore non generasse più errori del necessario. Questo linguaggio sarebbe ovviamente fortemente tipato.

Questo linguaggio però non può esistere. Se esistesse il compilatore per essere capace di decidere la correttezza dell'ultima assegnazione in un caso del tipo:

```
int x;
P;
x = " pippo";
```

Dove P è un generico programma, dovrebbe essere capace di decidere la terminazione di P (l'istruzione errata eseguita se e solo se la chiamata a P termina).

Pertanto questo linguaggio non sarebbe Turing completo, contro l'ipotesi.

I compilatori nella situazione precedente ignorano se una istruzione viene eseguita o meno e segnalano comunque un errore nell'ultima linea.

```
program wrong (input, output)
var i: integer;
begin
    if false then i:=3.14;
    else i :=0;
end.
```

5.1.5 Strategie

La strategia per i linguaggi fortemente e staticamente tipati è:

- Fare più controlli possibili a tempo di esecuzione;
- Eseguire i restanti a tempo di esecuzione.

Prima si scoprono gli errori e più il testing diventa economico.

5.2 Blocchi di istruzioni

Istruzioni raggruppate in blocchi per meglio definire:

- Ambito delle strutture di controllo;
- Ambito di una procedura;
- Unità di compilazione separata;
- Ambito dei legami di nome;

I blocchi che definisco l'ambito di validità di un nome contengono due parti:

- Sezione di dichiarazione del nome;
- Sezione che comprende gli enunciati sui quali ha validità il legame.

```
...
BLOCK A;
    DECLARE I;
BEGIN A
...
END A
```

5.2.1 Ambito di validità di legami

Essenzialmente due tipi di ambito di validità (scoping):

- ambito statico o lessicale (linguaggi moderni)
  - blocchi annidati vedono e usano i legami dei blocchi più esterni (di solito aggiungon legami locali o sovrapporne di nuovi)
- ambito dinamico (linguaggi come lisp)
  - assume il proprio senso maggiore quando vi sono procedure chiamanti e chiamate (la chiamata vede e usa i legami visti e usati dalla procedura chiamante)

5.2.2 Legami di nome

```
PROGRAM P;
    DECLARE X;
BEGIN P
... {X da P}
    BLOCK A;
        DECLARE Y;
    BEGIN A
    ... {X da P, Y da A}
        BLOCK B;
            DECLARE Z;
        BEGIN B
        ... {X da P, Y da A, Z da B}
        END B;
```

```
        ... {X da P, Y da A}
    END A;
    ... {X da P}
    BLOCK C;
        DECLARE Z;
    BEGIN C
    ... {X da P, Z da C}
    END C;
... {X da P}
END P;
```

5.2.3 Mascheramento

```
PROGRAM P;
    DECLARE X, Y;
BEGIN P
... {X e Y da P}
    BLOCK A;
        DECLARE X, Z;
    BEGIN A
    ... {X e Z da P, Y da P}
    END A;
    ... {X e Y da P}
END P;
```

5.2.4 Legami di locazione

Si dice allocazione statica quando le variabili conservano il proprio valore ogni volta che si rientra in un blocco (il legame di locazione è fissato e costante al tempo di caricamento)

```
PROGRAM P;
    DECLARE I;
BEGIN P
    FOR I :=1 TO 10 DO
    BLOCK A;
        DECLARE J;
    BEGIN A
        IF I=1 THEN
            J:=1; {I da P, J da A}
        ELSE
```

```
        J:= J*I;
        END IF
    END A;
END P;
```

Si dice allocazione dinamica di memoria quando il legame di locazione (e anche di nome) è creato all'inizio dell'esecuzione di un blocco e viene rilasciato a fine esecuzione.

È realizzata attraverso il record di attivazione di un blocco → contiene tutte le informazioni sull'esecuzione del blocco necessarie per riprendere l'esecuzione dopo che essa è stata sospesa.

Il record di attivazione puà contenere informazioni complesse, ma per realizzare un legame dinamico di locazione in blocchi annidati è sufficiente che contenga le locazioni dei dati locali più un puntatore al record di attivazione del blocco immediatamente più esterno

5.2.5 Stack di attivazione

In ogni momento dell'esecuzione lo stack di attivazione contiene i record "attivi":

- Il top dello stack contiene sempre il record del blocco correntemente in esecuzione;
- Ogni volta che si entra in un blocco, il record di attivazione del blocco viene posto sullo stack (push);
- ogni volta che si esce da un blocco, viene eliminato il record al top dello stack (pop).

es

```
PROGRAM P;
    DECLARE I, J;
    BEGIN P
        BLOCK A;
            DECLARE I,K;
        BEGIN A
            BLOCK B;
                DECLARE I,L;
            BEGIN B
                ... {I e L da B, K da A, J da P}
            END B;
            ... {I e K da A, J da P}
        END A;
        BLOCK C
            DECLARE I, N;
```

```
        BEGIN C
            ... {I e N da C, J da P}
        END C;
        ... {I e J da P}
END P
```

6 Lezione del 19-03

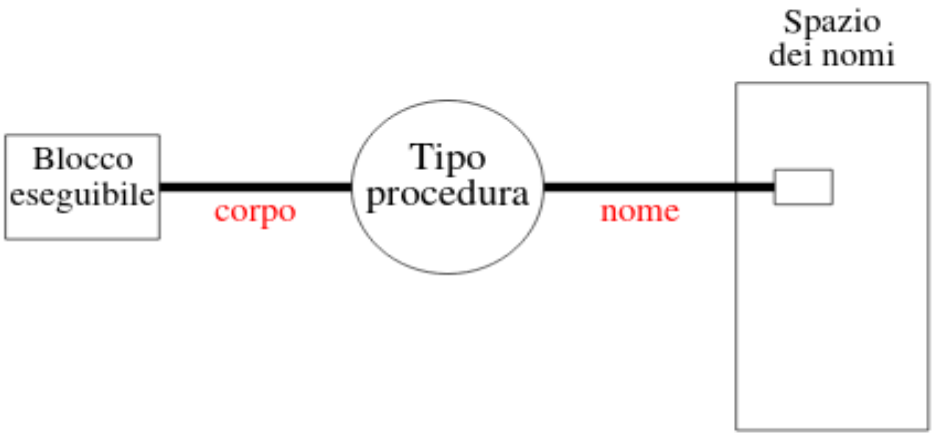
6.1 Procedure come astrazioni

Sono astrazioni di parti di programma in unità di esecuzione più piccole, come enunciati o espressioni, in modo da nascondere i dettagli ai fini del loro riuso Vantaggi:

- programmi più facili da scrivere, leggere e modificare;
- Unità di programmi indipendenti o con dipendenze benspecificate a livello più alto;
- Riusabilità del codice.

Se si distinguono come unità di esecuzione, in ordine crescente di complessità, espressioni, enunciati, blocchi, programmi, allora si definisce:

- astrazione procedurale la rappresentazione di una unità diesecuzione attraverso un'altra unità più semplice;
- Rappresentazione di un blocco attraverso un enunciato o un espressione;
- Causa la generazione di un oggetto analogo al Type Object;
- L'invocazione avviene durante l'esecuzione;
- Ogni invocazione diversa della stessa procedura causa lagenerazione di un nuovo "oggetto procedura" con lo stesso legame di tipo, ma con diverso record di attivazione.



6.2 Record di attivazione

Il record di attivazione rappresenta l'intero ambiente di esecuzione di una procedura. Esso consiste di solito in:

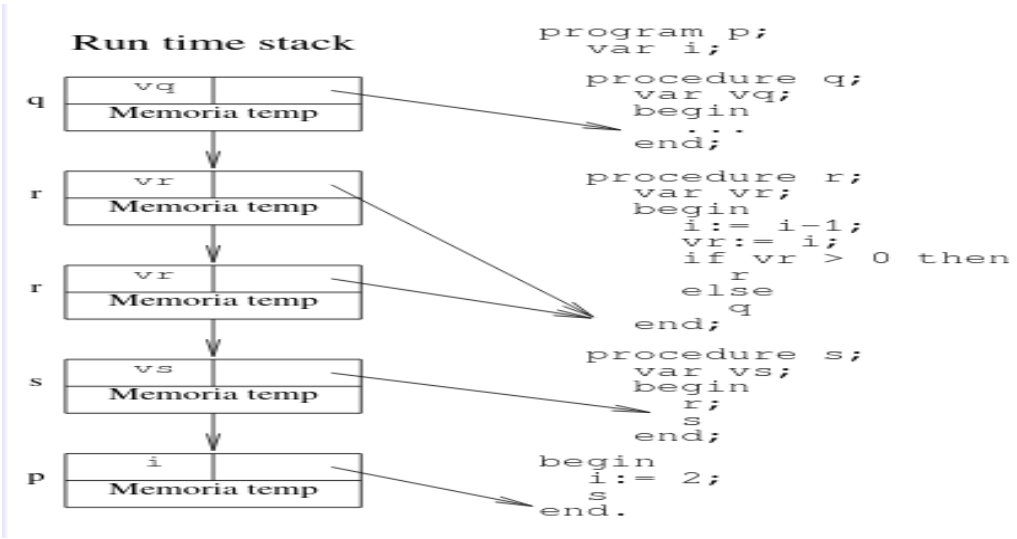
1. ambiente locale (tutti i data object che sono definiti all'internodella procedura);
2. ambiente non locale (tutti i data object la cui definizione'epropagata da altre procedure);
3. ambiente dei parametri (contiene informazioni sui dati che sonopassati [d]alla procedura);

Ogni volta che una procedura viene invocata il suo record di attivazione viene aggiunto al cosiddetto stack di esecuzione. Sul top dello stack c'è sempre il record relativo alla procedura correntemente in esecuzione. Alla terminazione della procedura, il record di attivazione viene rimosso dallo stack.

6.2.1 Ambiente locale

Include:

- Tutte le variabili dichiarate localmente;
- Il puntatore alla prossima istruzione;
- Memoria temporanea necessaria alla valutazione delle espressioni contenute nella procedura (altamente dipendente dalla realizzazione)



6.2.2 Propagazione dei Data OBJECT

Viene realizzato aggiungendo al record di attivazione un puntatore al record di attivazione della procedura da cui vengono propagate le definizioni o i dati.

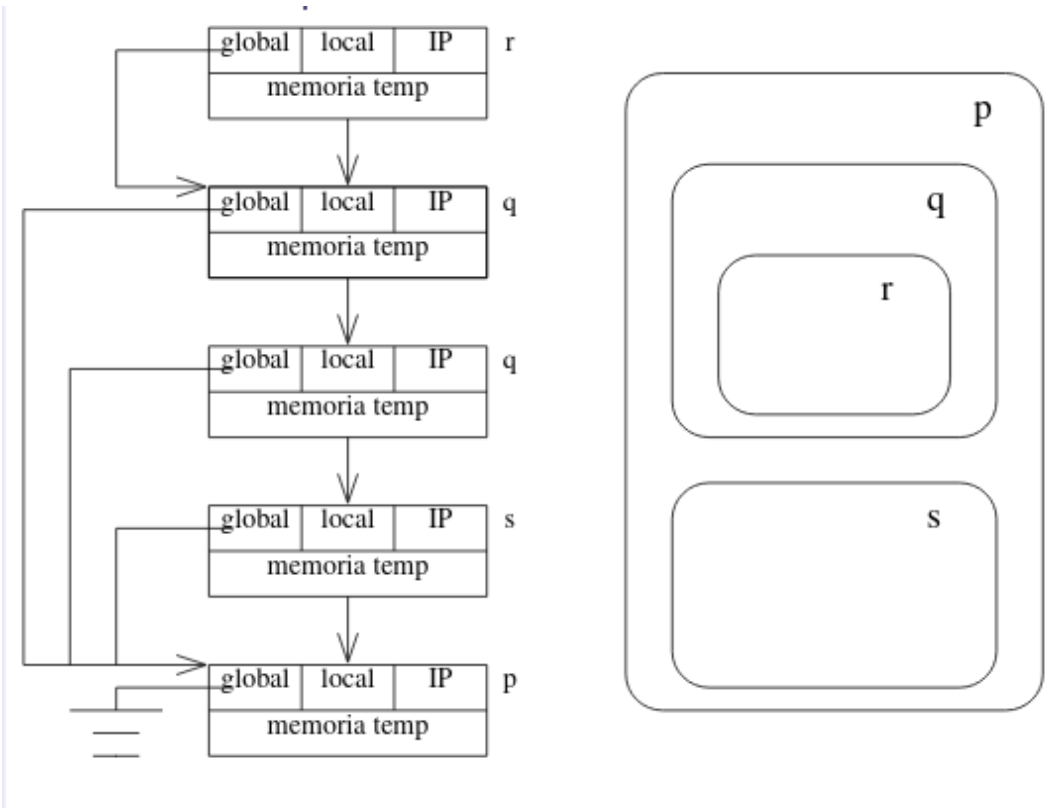
Se viene richiesto l'accesso ad un dato che non è definito localmente esso viene ricercato in modo ricorsivo nei record di attivazione precedenti. Tre tipologie di realizzazioen:

- Propagazione in ambito statico;
  - l'ambiente nonlocale di una procedura è propagato dal programma che la contiene sintatticamente → propagazione di posizione.
- Propagazione in ambito dinamico;
  - l'ambiente nonlocale di una procedura è propagato dal programma chiamante
- Nessuna Propagazione
  - l'uso di ambienti non locali è scoraggiato perchè produce effetti collaterali non facilmente prevedibili

Propagazione in ambito statico

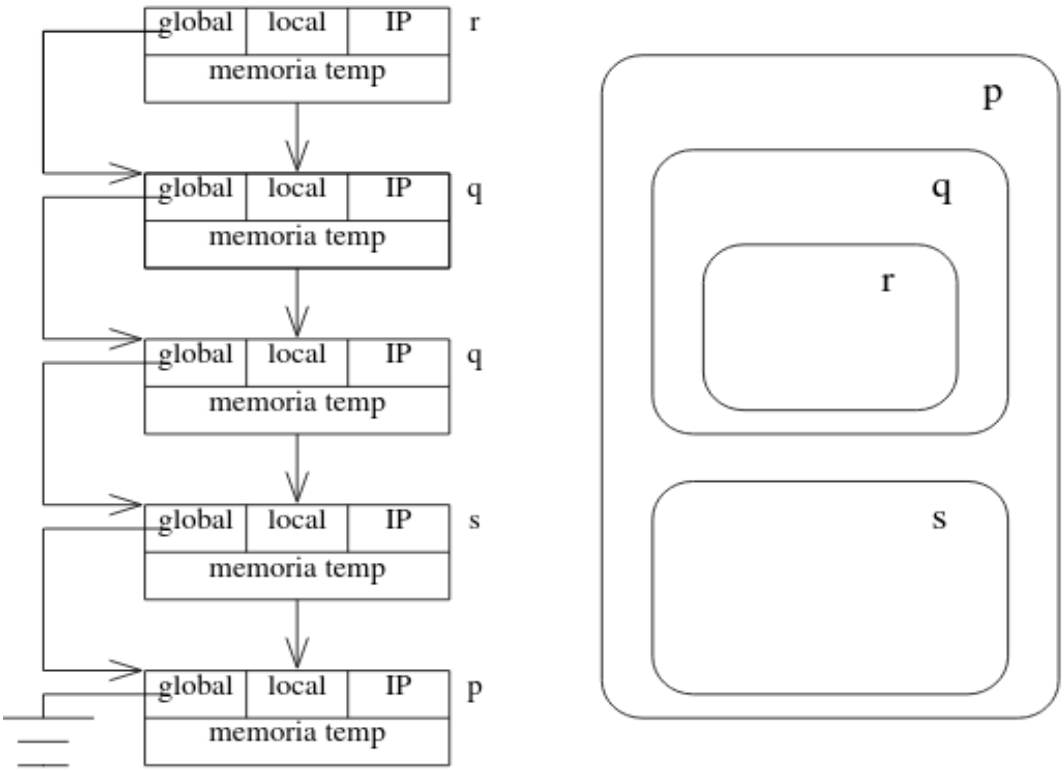
```
program p;
  var a, b, c: integer;

  procedure q;
    var a, c; integer;
    procedure r;
      var a: integer;
      begin {r}          {variabili: a da r; b da p; c da q;
        ...               procedure: q da p; r da q}
      end;  {r}
    begin {q}            {variabili: a da q; b da p; c da q;
       ...                procedure: q ed s da p; r da q}
    end;  {q}

  procedure s;
    var b: integer;
    begin {s}            {variabili: a da p; b da s; c da p;
      ...                 procedure: q ed s da p}
    end;  {s}

begin {p}                {variabili: a, b, c da p;
  ...                     procedure: q, s da p}
end.  {p}
```

Supponendo una sequenza di attivazione (p,s,q,q,r), lo stack di esecuzione ha questa forma:



Ambito dinamico

La stessa sequenza di attivazione precedente (p,s,q,q,r), generaallora lo stack di esecuzione:



Nella propagazione in ambito dinamico

- Il puntatore all'ambiente non locale non è più necessario;
- È praticamente impossibile la determinazione dell'ambiente di esecuzione di una procedura durante la scrittura del codice sorgente.

6.3 Parametrizzazione di procedure

Si distinguono in:

- parametri IN → sono passati dall'unità chiamante all'unità chiamata al momento dell'invocazione;
- parametri OUT → sono passati dall'unità chiamata alla unità chiamante al momento della terminazione della prima;
- parametri IN-OUT → transitare le informazioni in entrambe le direzioni.

Devono essere specificati in due punti:

- definizione della procedura → *parametri formali*;

- invocazione della procedura → *parametri attuali.*

Nella definizione deve essere specificato il tipo dei parametri formali. Nella invocazione è richiesta la corrispondenza di tipo tra parametri formali e attuali.

Eccezione comune:

- lasciare i parametri formali senza alcun legame di tipo → il legame si instaura durante l'esecuzione (run time) allo stesso tipo dei parametri attuali (impossibile il type checking in compilazione);
- array a dimensione variabile; il legame di tipo (e l'eventuale controllo di consistenza) verrà realizzato durante l'esecuzione.

Metodi di associazione:

- *per posizione* → a seconda della posizione relativa nella sequenza dei parametri;
- *per nome* → il nome del parametro formale è aggiunto come prefisso al parametro attuale;

es

procedure TEST (A: in Atype; b: in out Btype; C: out Ctype) – invocazione che usi associazione per posizione è: TEST(X, Y, Z); – mentre una che usi associazione per nome può essere TEST(A=>X, C=>Z, b=>Y);

Una tecnica è la cosidetta *associazione di default.* Essa permette di specificare valori di default ai parametri formali che non sono stati legati a valori da parametri attuali.

6.3.1 Parametri IN

Realizzati in 2 modi:

- Con un riferimento → la locazione del parametro attuale diventa la locazione del parametro formale
  - poichè il parametro formale è di tipo IN, allora si deve impedire la modifica all'interno della procedura;
- Con una copia → si alloca il parametro formale all'interno dello stack di attivazione
  - parametro formale e attuale presentano sono allocate in due posizioni diverse;
  - modifica permessa, perchè valida solo nell'ambiente diesecuzione della procedura;

Il secondo modo è meno efficiente del primo, sia rispetto allo spazio sia al tempo, ma è più flessibile e richiede meno variabili locali.

6.3.2 Parametri OUT

Realizzati in 2 modi:

1. Con un riferimento;
2. Con una copia.

Rappresentano risultati → alcuni linguaggi assumono che i parametri OUT non siano inizializzati e ne proibiscono la "lettura", ad es:

- Uso a destra di un assegnamento;
- Passaggio a un parametro IN o IN-OUT di un'altra procedura.

6.3.3 Parametri IN-OUT

Combinazione dei due precedenti. Realizzati in 2 modi:

- Con un riferimento;
- Con una copia.

6.4 Aliasing

È la possibilità di riferirsi alla stessa locazione con nomi diversi. Nel passaggio dei parametri può causare notevoli problemi di interpretazione.

6.5 Procedure come parametri di procedure

Alcuni linguaggi permettono l'uso di procedure come argomento di altre procedure.

7 Lezione del 23-03

7.1 Memo

In ADA sono presenti le keyword *in, out, in out.*

Pascal o non mette nulla (passaggio per copia) o utilizza la keyword *var* (*in out* per riferimento)

C, C++, Java presentano solo il passaggio *in* per copia

In C e C++ si può simulare *in-out* per riferimento (sfruttando gli opportuni operatori)

In Java il passaggio di oggetti somiglia a un passaggio per riferimento (viene passato il puntatore all'oggetto).

7.2 Funzioni

Sono procedure che restituiscono un valore alla procedura chiamante. Sono realizzate:

- O creando una pseudovariabile nell'ambiente locale della procedura chiamata
  - questa variabile può essere solo modificata (nessun accesso in lettura);
- O utilizzando una istruzione di return per restituire esplicitamente il controllo alla procedura chiamante inviandole allo stesso tempo il valore di una espressione

8 Lezione del 26-03

8.1 Esempi sul record di attivazione nei linguaggi

8.1.1 Esempio interferenza

```
#include <stdio.h>
void interferenza(int x){
    int var;
    printf("var = %d\n", var);
    var = x;
    }

void interferenza_bis(int x){
    int var;
    printf("var = %d\n", var);
    var = x;
}
```

```
int main (void){
    interferenza(1);
    interferenza_bis(2);
    interferenza(3);
}
```

$$\begin{aligned} \text{var} &= 21851 \\ \text{var} &= 1 \\ \text{var} &= 2 \end{aligned}$$

8.1.2 Esempio copia

```
#include <stdio.h>
void copy(char x){
  char buf[5];
  int i=0;
  do{
  buf[i] = x[i];
  }
  while(x[i++] != '\0');
}
int main(void){
    char v[] = "abcdef";
    copy(v);
    return 0;
}
```

Si ottiene uno `stack mashing detected` (si cerca di inserire dati in quantità maggiore di quanti ne possa contenere il `buf[5]`)

8.2 Macro

Generazione di un nuovo brano di codice sorgente (espansione della macro) in cui i nomi dei parametri attuali sostituiscono i nomi dei parametri formali. *es.* Data la procedura

```
procedure swap (a, b: integer);
var temp: integer;
begin
    temp := a;
    a := b;
```

```
    b := a;
end;
```

La chiamata *swap(x,y)* esegue il seguente brano di codice:

```
temp := x;
x := y;
y := temp;
```

8.3 Implemantazione efficiente dello scoping statico

L'accesso ad una variabile non è istantanea → bisogna scandire una lista che potrebbe essere lunga.

8.4 Annidamento

```
program p;
var a,b,c int;
    procedure q;
    var a,c int;
        procedure r;
        var a int;
    procedure s;
    var b int;
```

Una procedura q è annidata in un blocco b se è definita dentro b, ad *es*:

- q e s sono annidate in p;
- r è annidata in q.

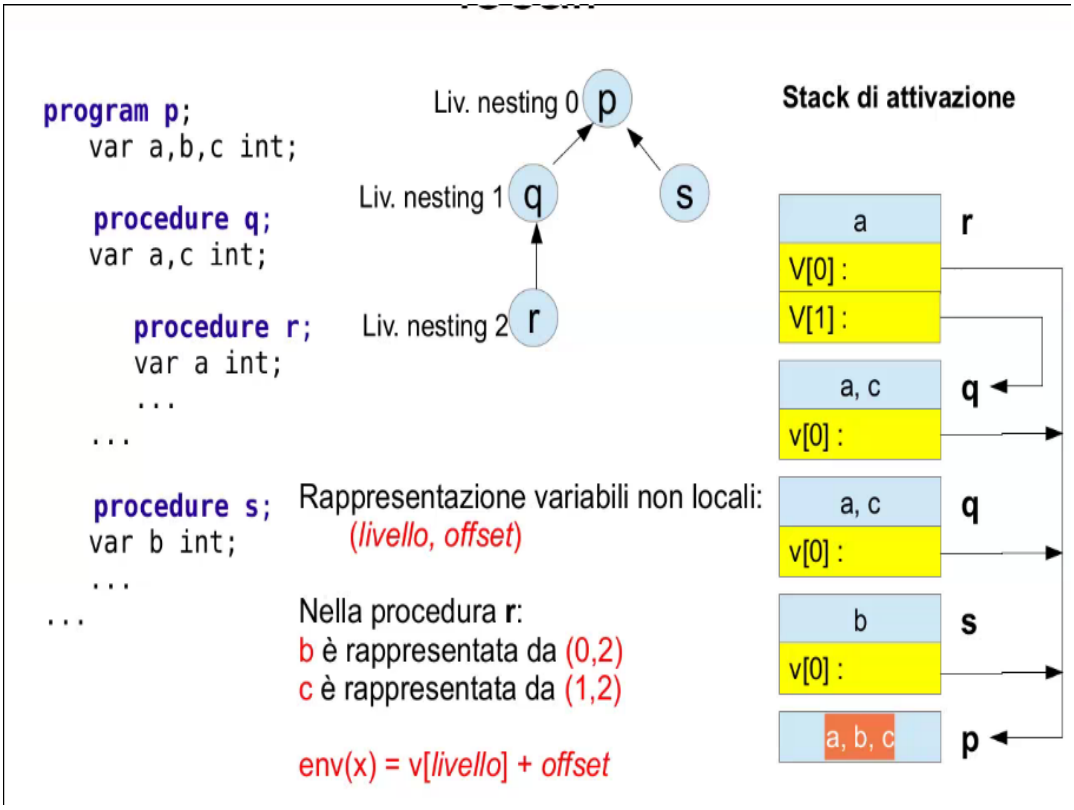
Il livello di nesting di una procedura è il numero di blocchi che a contengono:

- il livello di nesting di q ed s è 1;
- Il livello di nesting di r è 2.

L'ambiente statico non locale di una procedura è dato dall'ambiente delle procedure in cui è innestata

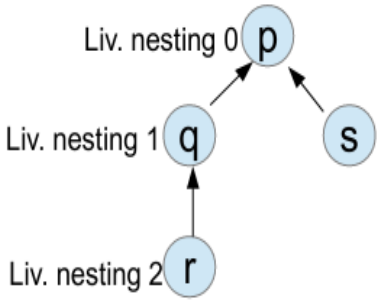
L'annidamento si può rappresentare come un albero. I livelli dell'albero di nesting corrispondono ai livelli di nesting

8.5 Rappresentazione variabili non locali



8.6 Mantenimento del vettore degli ambienti non locali

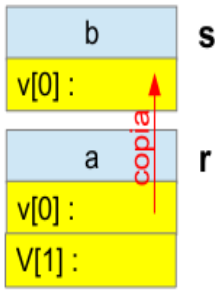
```
program p;
   var a,b,c int;

   procedure q;
   var a,c int;

      procedure r;
      var a int;
      ...
   ...

   procedure s;
   var b int;
   ...
...
```



Chiamata a procedure definite esternamente:

Esempio 2: r chiama s:

- il liv. di nesting **decresce**
- Copiare solo gli elementi di livello *minore o uguale* a quello di s
- Struttura ad albero + scoping statico garantiscono sempre di trovare l'ambiente non locale della procedura chiamata nel record del chiamante



8.7 Proprietà di questa implementazione

- Accesso alle variabili in tempo costante
  - Accesso a vettore + somma del puntatore ivi contenuto e dell'offset;
  - Indipendente dal livello di nesting;
  - Calcolo supportato direttamente dalle istruzioni macchina.
- Creazione record di attivazione lineare nel livello di nesting (copia degli elementi di v[])
  - Indipendente dall'esecuzione;
  - Dipende solo dal testo del sorgente;
- Tempo costante;
- L'implementazione naive richiederebbe a run time di percorrere le liste di puntatori all'ambiente non locale
  - Accesso alle variabili in tempo **lineare** nel livello di nesting;
  - Creazione dei record di attivazione in tempo **lineare** nel livello di nesting.

8.8 Paradigma OO

Rappresentazione di ogni cosa in un programma OO.

In essenza → progettare un sistema OO consiste nell'identificare quali oggetti il sistema debba contenere, il loro comportamento e le responsabilità di ognuno di essi e come essi interagiscono tra loro

8.9 Encapsulation

Le strutture dati che implementano i valori sono accessibili direttamente solo dalle operazioni fornite dal tipo → è una regola di **scoping**

8.10 C++ vs Java

C++

Acquisisce eredità di C ed è compatibile con programma C esistenti.

Cresciuto in modo abnorme ed è veramente difficile raggiungere piena competenza in tutti gli aspetti del linguaggio.

Java

WWW e abilità di eseguire *web applet* su qualunque elaboratore e SO.

Linguaggio ben progettato con pochi costrutti. Risolti i prooblemi della velocità di esecuzione.

Diffusamente utilizzato nell'ambito educativo

8.11 UML

Linguaggio i cui simboli sono diagrammi

8.12 Essenza di un oggetto

- Insieme di **attributi** (valore, stato interno, o qualunque altra cosa sia necessaria per il modello dell'oggetto);
- **Abilità** di modificare lo stato degli attributi;
- **Responsabilità** sotto forma di servizi offerti ad altri oggetti.

  Gli oggetti esterni non hanno conoscenza di come un oggetto realizza gli attributi internamente, ma piuttosto devono conoscere quali servizi un oggetto offre.

8.13 Classe

Descrizione di un insieme di oggetti che condivdono attributi e comportamento comuni. Simile a quello del tipo di dato astratto nei linguaggi non OO

La definizione di una classe descrive tutti gli attributi comuni agli oggetti della classe, così come tutti i metodi che realizzano il comportamento comune degli oggetti della classe.

Gli oggetti membri di una classe sono chiamati **istanze** di quella classe → I membri di una classe variano durante l'esecuzione di un programma

8.14 Polimorfismo

Permette al mittente di ignorare l'identità (il tipo) del ricevente.

Realizzato con un mix di overriding e upcasting.

9 Lezione del 30-03

9.1 Classi astratte

Definiscono una interfsaccia comune a tutte le sottoclassi, specificando solo i nomi (la firma) dei metodi che le sottoclassi "concrete" dovranno definire.

Poichè sono incomplete della realizzazione dei metodi, le classi astratte non possono avere istanze.

Una classe che estende una classe astratta eredita tutti i suoi metodi (inclusi quelli astratti). Ogni classe che eredita metodi astratti è considerata astratta essa stessa, a meno che essa nonrealizzi tutti i metodi astratti ereditati

9.2 Visibilità delle componenti di una classe

È la possibilità di una classe di veere ed usare le risorse di un'altra classe. Livello di visibilità:

- private (`-`) → attributi e metodi sono visibili solo a oggetti istanze della stessa classe;
- Package (`#`) → attributi e metodi sono visibili ad un insieme specifico di classi;
- Public (`+`) → attributi e metodi sono visibili a tutti gli altri oggetti.

Normalmente si cerca di evitare attributi publici. Se si deve reperire o modificare le informazioni sullo stato interno di uno oggetto, lo si fa utilizzando i metodi specifici (e visibili) dell'oggetto

9.3 Ereditarietà

Il concetto di ereditarietà descrive come si possano condividere attributi e funzionalità tra classi di natura o scopo simile.

Vi sono due modi con cui si può aggiungere il concetto di ereditarietà al modello del sistema:

- Generalizzazione;
- Specializzazione.

9.3.1 Generalizzazione

Avviene quando si riconosce che più classi dello stesso diagramma Class esibiscono funzionalità,struttura, scopo comuni.L'analista decide allora di creare una nuova classe contenente gli attributi e le funzionalità comuni esemplifica le classi precedenti come estensioni della nuova(generalizzata).

9.3.2 Specializzazione

Avviene quando si riconosce che la nuova classe che si sta aggiungendo ha le stesse funzionalità,struttura e scopo di una classe esistente, ma ha bisogno di nuovo codice o attributi.La nuova classe è una forma *specializzata* di quella già esistente.

9.4 Associazioni e molteplicità

9.4.1 Associazioni complesse

Sono quelle associazioni in cui vi sono molteplicità maggiori di 1 in entrambe i ruoli. Difficile da implementare. 2 possibili soluzioni:

- Classe di associazione
  - Il nome della classe deve essere lo stesso della associazione;
  - Una classe di Associazione è usata per documentare la risoluzione di una associazione complessa, non fa parte delle astrazioni chiave;
  - Restano nei diagrammi Class finchè, nella fase di progettazione, non si decide di realizzarle;
- Associazione qualificata
  - Usate quando si vuole evidenziare che la realizzazione futura sarà mediante un array, una hash table, un dizionario;

9.4.2 Ruoli nelle associazioni

- Riferiscono ad una associazione tra classi;

- Etichette poste alle estremità di una associazione;
- Rappresentano un attributo all'interno della classe che si trova all'estremità opposta del nome.

9.4.3 Associazioni riflessive

In un diagramma Object possono esistere link tra istanze della stessa classe, ma c'è una sola classe nel diagramma Class per rappresentare entrambi gli oggetti. In tal caso è usata una associazione riflessiva.

9.5 Introduzione a Java

Un linguaggio di programmazione, con sintassi simile a C++ e semantica simile a SmallTalk.

Le applicazioni Java sono programmi autonomi che richiedono un elaboratore su cui sia installato Java Runtime Environment(JRE).

Un ambiente di sviluppo, cioè un insieme di numerosi strumenti per la realizzazione di programmi: un compilatore, un interprete, un generatore di documentazione, uno strumento di aggregazione di file, etc...

9.5.1 Errori comuni

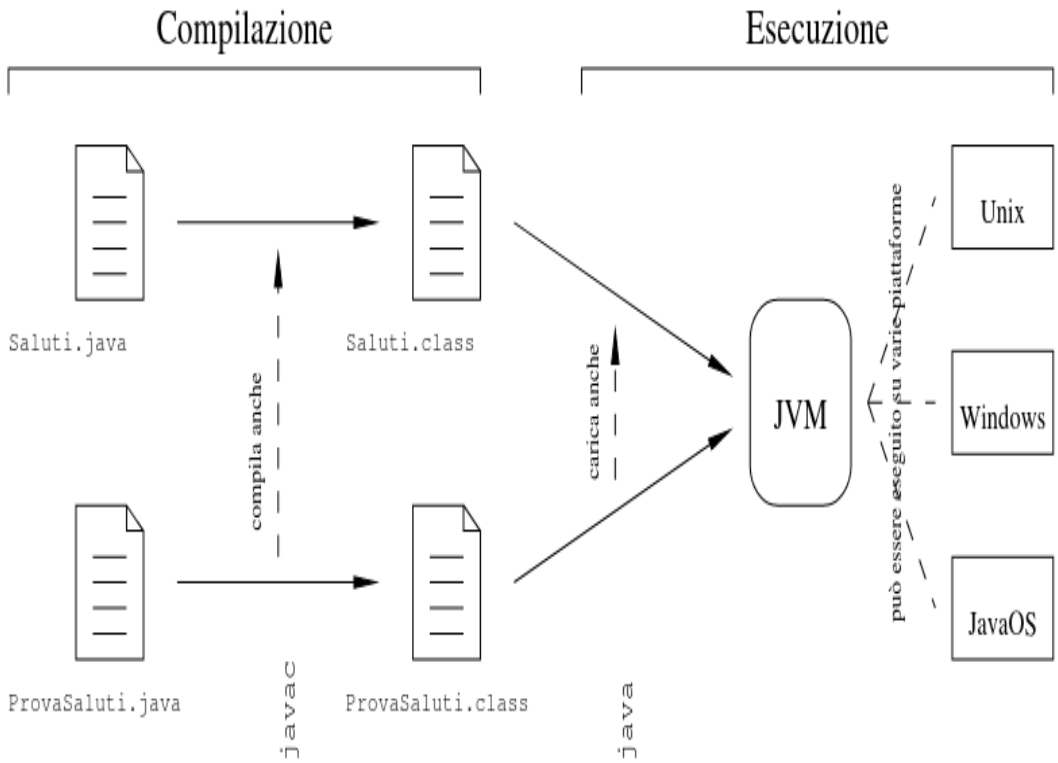
Errori in compilazione:

- Installazione JavaSDK non corretta;
- Nome di metodi di sistema non corretti;
- Errata corrispondenza tra nome di classe e nome di file;
- Numero di classi pubbliche in un file;

Errori in esecuzione:

- Errata scrittura della classe da caricare inizialmente in memoria;
- Errata scrittura della firma del metodo main;

9.5.2 Compilazione ed Esecuzione



9.5.3 Sicurezza

Miglioramento della sicurezza del codice che proviene da fonti esterne.

Il Class Loader:

- Carica solo le classi necessarie per l'esecuzione del programma;
- Mantiene le classi del file system locale in uno spazio di nomi separato
  - limita applicazioni "cavallo di Troia", poichè le classi locali sono sempre caricate per prime;
- previene il così detto *spoofing*.

Il Bytecode Verifier controlla che:

- Il codice rispetti le specifiche della JVM;
- Il codice non violi l'integrità del sistema;
- Il codice non generi stack overflow o underflow;
- Non vi siano conversioni di tipo illegali (*es* la conversione di interi in puntatori).

9.5.4 Costruttori

NB:

- I costruttori NON sono metodi;
- I costruttoni NON sono ereditati;
- Un costruttore NON ha valore di ritorno;
- Gli unici modificatori validi per il costruttore sono public,protected e private.

Costruttore di default

C'è sempre almeno un costruttore in ogni classe. Se il programmatore non scrive nessun costruttore, allora verrà usato un costruttore di default.

Il costruttore di default non ha argomenti, non ha corpo.

Permette di creare istanze di classi (con l'espressionenew Xxx()) senza dover scrivere un costruttore.

NB → Se si aggiunge una dichiarazione di costruttore con argomenti ad una classe che in precedenza non aveva costruttori espliciti, allora si perde il costruttore di default. Da quel punto in poi, una chiamata a *new Xxx()* genererà un errore di compilazione.